```
import numpy as np
one = [0,0,0,0,1,0,0,0,0]
two = [0,0,1,0,0,0,1,0,0]
thr = [0,0,1,0,1,0,1,0,0]
fou = [1,0,1,0,0,0,1,0,1]
fiv = [1,0,1,0,1,0,1,0,1]
six = [1,0,1,1,0,1,1,0,1]
dice = np.array([one, two, thr, fou, fiv, six])
```

```
dice
```

```
array([[0, 0, 0, 0, 1, 0, 0, 0, 0],
       [0, 0, 1, 0, 0, 0, 1, 0, 0],
       [0, 0, 1, 0, 1, 0, 1, 0, 0],
       [1, 0, 1, 0, 0, 0, 1, 0, 1],
       [1, 0, 1, 0, 1, 0, 1, 0, 1],
       [1, 0, 1, 1, 0, 1, 1, 0, 1]])
```

How does this represent a die?

```
0, 0, 0, 0, 1, 0, 0, 0, 0
```

Well, we think of each set of 3 values as a row (of 3 total rows). We think of `1` as the presence of a dot, and `0` as the absence a dot.

So, the above collection of 9 values would represent a die that looks like this:

0	0	0
0	1	0
0	0	0

Or if we just have a dot…

O

What is element-wise multiplication?

```
dice[4] * dice[5]
```

```
array([1, 0, 1, 0, 0, 0, 1, 0, 1])
```

```
is_one = np.array([-1,-1,-1,-1,1,-1,-1,-1,-1])
```

```
for die in dice:
  print(1 <= sum(is_one * die))
```

```
True
False
False
False
False
False
```

```
(0 < np.sum(dice * is_one, 1)) + 0
```

```
array([1, 0, 0, 0, 0, 0])
```

This is what we would want to see when examining a `fiv`.

```
[0,0,0,0,1,0]
```

```
[0, 0, 0, 0, 1, 0]
```

```
multi = np.array([
    [-1/1, -1/1, -1/1, -1/1,  1/1, -1/1, -1/1, -1/1, -1/1],
    [-1/2, -1/2,  1/2, -1/2, -1/2, -1/2,  1/2, -1/2, -1/2],
    [-1/3, -1/3,  1/3, -1/3,  1/3, -1/3,  1/3, -1/3, -1/3],
    [ 1/4, -1/4,  1/4, -1/4, -1/4, -1/4,  1/4, -1/4,  1/4],
    [ 1/5, -1/5,  1/5, -1/5,  1/5, -1/5,  1/5, -1/5,  1/5],
    [ 0/6,  3/6,  0/6,  3/6, -6/6,  3/6,  0/6,  3/6,  0/6],
])
```

```
fiv
```

```
[1, 0, 1, 0, 1, 0, 1, 0, 1]
```

```
fiv * multi
```

```
array([[-1.        , -0.        , -1.        , -0.        ,  1.        ,
        -0.        , -1.        , -0.        , -1.        ],
       [-0.5       , -0.        ,  0.5       , -0.        , -0.5       ,
        -0.        ,  0.5       , -0.        , -0.5       ],
       [-0.33333333, -0.        ,  0.33333333, -0.        ,  0.33333333,
        -0.        ,  0.33333333, -0.        , -0.33333333],
       [ 0.25      , -0.        ,  0.25      , -0.        , -0.25      ,
        -0.        ,  0.25      , -0.        ,  0.25      ],
       [ 0.2       , -0.        ,  0.2       , -0.        ,  0.2       ,
        -0.        ,  0.2       , -0.        ,  0.2       ],
       [ 0.        ,  0.        ,  0.        ,  0.        , -1.        ,
         0.        ,  0.        ,  0.        ,  0.        ]])
```

```
np.sum(fiv * multi, 1)
```

```
array([-3.        , -0.5       ,  0.33333333,  0.75      ,  1.        ,
       -1.        ])
```

```
(np.sum(fiv * multi, 1) >= 1) + 0
```

```
array([0, 0, 0, 0, 1, 0])
```

```
for die in dice:
  print((1 <= np.sum(die * multi, 1)) + 0)
```

```
[1 0 0 0 0 0]
[0 1 0 0 0 0]
[0 0 1 0 0 0]
[0 0 0 1 0 0]
[0 0 0 0 1 0]
[0 0 0 0 0 1]
```

```
class Neuron:

    def __init__(self, weights, bias):
        self.weights = np.array(weights)
        self.bias = bias
```

```
    def __repr__(self):
        return "Neuron" + str(self.weights)

    def __mul__(self, other):
        return int(np.sum(self.weights * other) >= self.bias)
```

```
is_one_neuron = Neuron(is_one, 1)
```

```
for die in dice:
  print(1 <= sum(is_one * die))
```

```
True
False
False
False
False
False
```

```
for die in dice:
  print(is_one_neuron * die)
```

```
1
0
0
0
0
0
```

```
is_fiv_neuron = Neuron([ 1/5, -1/5,  1/5, -1/5,  1/5, -1/5,  1/5, -1/5,  1/5], 1)
print(is_fiv_neuron)
```

```
Neuron[ 0.2 -0.2  0.2 -0.2  0.2 -0.2  0.2 -0.2  0.2]
```

```
for die in dice:
  print(is_fiv_neuron * die)
```

```
0
0
```

```
0
0
1
0
```

```
multi
```

```
array([[-1.        , -1.        , -1.        , -1.        ,  1.        ,
        -1.        , -1.        , -1.        , -1.        ],
       [-0.5       , -0.5       ,  0.5       , -0.5       , -0.5       ,
        -0.5       ,  0.5       , -0.5       , -0.5       ],
       [-0.33333333, -0.33333333,  0.33333333, -0.33333333,  0.33333333,
        -0.33333333,  0.33333333, -0.33333333, -0.33333333],
       [ 0.25      , -0.25      ,  0.25      , -0.25      , -0.25      ,
        -0.25      ,  0.25      , -0.25      ,  0.25      ],
       [ 0.2       , -0.2       ,  0.2       , -0.2       ,  0.2       ,
        -0.2       ,  0.2       , -0.2       ,  0.2       ],
       [ 0.        ,  0.5       ,  0.        ,  0.5       , -1.        ,
         0.5       ,  0.        ,  0.5       ,  0.        ]])
```

```
multi.shape
```

```
(6, 9)
```

Six classes; nine possible observation sites.

```
layer = np.array([
    Neuron([-1/1, -1/1, -1/1, -1/1,  1/1, -1/1, -1/1, -1/1, -1/1], 1),
    Neuron([-1/2, -1/2,  1/2, -1/2, -1/2, -1/2,  1/2, -1/2, -1/2], 1),
    Neuron([-1/3, -1/3,  1/3, -1/3,  1/3, -1/3,  1/3, -1/3, -1/3], 1),
    Neuron([ 1/4, -1/4,  1/4, -1/4, -1/4, -1/4,  1/4, -1/4,  1/4], 1),
    Neuron([ 1/5, -1/5,  1/5, -1/5,  1/5, -1/5,  1/5, -1/5,  1/5], 1),
    Neuron([ 0/6,  3/6,  0/6,  3/6, -6/6,  3/6,  0/6,  3/6,  0/6], 1),
])
```

```
layer
```

```
array([Neuron[-1. -1. -1. -1.  1. -1. -1. -1. -1.],
       Neuron[-0.5 -0.5  0.5 -0.5 -0.5 -0.5  0.5 -0.5 -0.5],
```

```
       Neuron[-0.33333333 -0.33333333  0.33333333 -0.33333333  0.33333333 -
0.33333333
         0.33333333 -0.33333333 -0.33333333]                                  ,
       Neuron[ 0.25 -0.25  0.25 -0.25 -0.25 -0.25  0.25 -0.25  0.25],
       Neuron[ 0.2 -0.2  0.2 -0.2  0.2 -0.2  0.2 -0.2  0.2],
       Neuron[ 0.   0.5  0.   0.5 -1.   0.5  0.   0.5  0. ]], dtype=object)
```

```
np.random.rand()
```

```
0.41110762714122984
```

```
learn = np.random.rand(6,9)
print(learn)
```

```
[[0.01164202 0.9340941  0.90124772 0.81454318 0.84100497 0.83022864
  0.60323278 0.94634327 0.41691854]
 [0.09878284 0.31585217 0.15939342 0.77342747 0.85789747 0.05476256
  0.48608471 0.1738215  0.46384816]
 [0.94127555 0.25587752 0.46294515 0.3907162  0.10431976 0.65622714
  0.23602205 0.58387894 0.97189415]
 [0.67008577 0.50211072 0.67384859 0.65351941 0.26808872 0.22730754
  0.63236681 0.08453789 0.95844623]
 [0.08393118 0.53360437 0.55565322 0.20027961 0.06168624 0.64217301
  0.36145554 0.57006812 0.07397591]
 [0.31548117 0.05953092 0.43680787 0.1605615  0.11777931 0.6367064
  0.3480973  0.1889091  0.8861156 ]]
```

```
np.random.rand(9).shape
```

```
(9,)
```

```
is_one.shape
```

```
(9,)
```

```
np.random.rand()
```

```
class Neuron:

    def __init__(self, weights=np.random.rand(9), bias=1):
        self.weights = np.array(weights)
        self.bias = bias

    def __repr__(self):
        return "Neuron" + str(self.weights)

    def __mul__(self, other):
        return int(np.sum(self.weights * other) >= self.bias)
```

```
learner = Neuron()
print(learner)
```

```
Neuron[0.64043152 0.015      0.25987457 0.65689431 0.13822314 0.00357436
 0.37411781 0.59799956 0.83062772]
```

```
Neuron(is_one)
```

```
Neuron[-1 -1 -1 -1  1 -1 -1 -1 -1]
```

```
rand_n = Neuron()
```

```
for die in dice:
  print(rand_n * die)
  print(is_fiv_neuron * die)
  print()
```

```
0
0

0
0

0
0

1
```

```
0

1
1

1
0
```

```
class Neuron:

    def __init__(self, weights=np.random.rand(9), bias=1):
        self.weights = np.array(weights)
        self.bias = bias

    def __repr__(self):
        return "Neuron" + str(self.weights)

    def __mul__(self, other):
        return int(np.sum(self.weights * other) >= self.bias)

    def learn(self, perceived, answer):
        if self * perceived == answer:
            temp = perceived + .5
            temp /= sum(perceived)
            self.weights *= temp
            return
        delta = self.bias - sum(self.weights * perceived)
        each = (delta * 1.1) / sum(perceived)
        update = each * perceived
        self.weights += update
```

```
n = Neuron()
answer_key = [0,0,0,0,1,0]
for i in range(6):
  n.learn(dice[i], answer_key[i])
```

```
for i in range(6):
  print(i + 1, n * dice[i])
```

```
1 0
2 0
```

```
3 0
4 0
5 0
6 0
```

```
for i in range(1000):
  i = np.random.randint(6)
  n.learn(dice[i], answer_key[i])

for i in range(6):
  print(i + 1, n * dice[i])
```

```
1 0
2 0
3 0
4 0
5 0
6 0
```

```
n
```

```
Neuron[0.00342979 0.         0.03102563 0.         0.00342983 0.
 0.03102563 0.         0.00342979]
```

```
is_fiv_neuron
```

```
Neuron[ 0.2 -0.2  0.2 -0.2  0.2 -0.2  0.2 -0.2  0.2]
```

```
class Neuron:

    def __init__(self, weights=np.random.rand(9), bias=1):
        self.weights = np.array(weights)
        self.bias = bias

    def __repr__(self):
        return "Neuron" + str(self.weights)

    def __mul__(self, other):
        return int(np.sum(self.weights * other) >= self.bias)
```

```
    def learn(self, perceived, answer):
        if answer == self * perceived:
            temp = perceived + .5
            temp /= sum(perceived)
            self.weights *= temp
        else:
            delta = self.bias - np.sum(self.weights * perceived)
            each = delta / sum(perceived)
            update = each * np.array(perceived)
            self.weights = self.weights + update
```

```
learner = Neuron()

for _ in range(1000):
    r = int(np.random.rand() * 6)
    learner.learn(dice[r], answer_key[r])
```

```
for i in range(6):
    print(i + 1, dice[i], answer_key[i], learner * dice[i])
```

```
1 [0 0 0 0 1 0 0 0 0] 0 0
2 [0 0 1 0 0 0 1 0 0] 0 0
3 [0 0 1 0 1 0 1 0 0] 0 0
4 [1 0 1 0 0 0 1 0 1] 0 0
5 [1 0 1 0 1 0 1 0 1] 1 1
6 [1 0 1 1 0 1 1 0 1] 0 0
```